WORK IN PROGRESS

Bachelor's Thesis

by

Jakob Meyerhöfer

(Mail: meyerhoe@rptu.de)

March 5, 2026

University of Kaiserslautern-Landau
Department of Computer Science
67663 Kaiserslautern
Germany

Examiner: Prof. Dr. Ralf Hinze
Cass Alexandru (Gregor Cassian Alexandru), M.Sc.

Declaration of Authorship

I hereby declare that the work I have submitted with the title „WORK IN PROGRESS“ was independently written by me. I have fully disclosed all sources and aids used and have identified as quotations all passages of the work — including tables and figures — taken verbatim or in substance from other works or the internet, indicating the respective sources.

Rodalben, on 5.3.2026

Jakob Meyerhöfer

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Summary

TODO

Contents

1 Introduction **1**
- 1.1 Problem Statement and Objectives . . . 2
- 1.2 Research Questions . . . 3
- 1.3 Thesis Structure . . . 3

2 Background and Related Work **5**
- 2.1 Challenges for Novice Programmers . . . 5
- 2.2 Error Messages and Programming Education . . . 5
- 2.3 Improving Feedback: Existing Aproches (Rust?) . . . 5

3 Problem Analysis **7**
- 3.1 Context and Assumptions . . . 7
- 3.2 Observed Issues in Existing Compiler/Test Output . . . 8
- 3.3 Derived Requirements and Design Implications . . . 10

4 Assertify (and Checkify): Design and Implementation **13**
- 4.1 Design Goals and Principles . . . 13
- 4.2 Output Model: Representing Tests . . . 13
- 4.3 Mechanisms: Expression Capture and Value Extraction . . . 13
- 4.4 Reporting: Formatting and Integration with the Test Pipeline . . . 13

5 Results and Discussion **15**
- 5.1 Research Question Coverage and Requirement Satisfaction . . . 15
- 5.2 Artifact Outcomes . . . 15
- 5.3 Key Findings and Interpretations . . . 15
- 5.4 Strengths and Limitations . . . 15

6 Conclusion **17**
- 6.1 Summary . . . 17
- 6.2 Future Work . . . 17
- 6.3 Final Remarks . . . 17

List of Figures **19**

List of Listings **21**

List of Tables **23**

My Code **25**

My Ideas **27**

1 Introduction

All programmers have to start at the beginning: What is programming? What is required to write and execute code? What are variables and methods and how are they used? From there, learners gradually encounter key concepts and algorithms, and later data structures such as maps and sets. However, moving from basic syntax to independently writing correct programs is a long process, which also includes implementing corresponding unit tests to validate solutions.

Understanding and interpreting error messages is a fundamental challenge for novice programmers. Error messages serve as an essential form of feedback, enabling programmers to diagnose issues in their code. In practice, standard diagnostics - especially those produced by compilers and testing frameworks - are often technical, verbose, and difficult for beginners to interpret.

This becomes particularly apparent in introductory programming courses where students receive exercise templates with predefined tests and are expected to implement missing functions or data structures. In such settings, learners are frequently unfamiliar with unit testing and are immediately confronted with stack traces and framework-specific output formats.

In F#, multiple testing frameworks and assertion libraries are available. A commonly used option is Microsoft.VisualStudio.TestTools.UnitTesting. For example, using `Assert.AreEqual` can yield the following output on failure:

```
Assert.AreEqual failed. Expected:<[1; 2; 4]>. Actual:<[1; 4; 2]>.
    at MapTests.TestsMapSortedList.Tests.testSetBeispiele() in /code/
    ↪  TestsMapSortedList.fs:line 53
    at System.RuntimeMethodHandle.InvokeMethod(Object target, Void**
    ↪  arguments, Signature sig, Boolean isConstructor)
    at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj,
    ↪  BindingFlags invokeAttr)
```

Error Message 1.1 Example Ouput of `Assert.AreEqual`

For more advanced programmers, this output is usually sufficient: they can quickly identify the relevant mismatch and ignore the remaining details. See Section TODO for a discussion of common issues in diagnostic messages and strategies for extracting key information.

In my experience as a tutor for a first-semester lecture, I frequently observe students struggling with such messages, unsure what went wrong and how to fix their code. Instead of guiding them toward a solution, poorly structured error messages can lead to frustration and trial-and-error debugging, which may hinder learning and delay progress on key programming concepts.

Didactically, complex concepts are typically introduced gradually, often by reducing incidental complexity at the beginning:

- Lightweight IDE's are often preferred to heavy, feature-rich IDE's
- Simplistic I/O is introduced before full input validation only valid inputs and error handling (e.g., malformed inputs, exceptions, and missing values).

The same Diagnostic verbosity can be staged for testing: Initially showing only the most relevant information and revealing additional technical details as students gain experience. This thesis explores how a similar scaffolding approach can be applied to testing feedback. By providing more minimalistic and structured output, the goal is to help students interpret failures more effectively and support a more efficient and engaging learning process.

1.1 Problem Statement and Objectives

The primary problem addressed in this thesis is that, in introductory programming settings, the feedback produced by common testing frameworks is often not presented in a form that is immediately interpretable for novice programmers. In particular, failure reports frequently combine the core mismatch information (i.e., what was expected versus what occurred) with framework-specific detail such as stack traces and internal method calls. As a consequence, beginners may struggle to identify the relevant part of the output and translate it into concrete corrective actions in their own code.

From a technical perspective, this raises the question of how test feedback can be reformatted such that it is **(i)** consistent and cognitively simple for first-semester students, while **(ii)** remaining compatible with established testing infrastructures. This thesis therefore investigates the design and implementation of a lightweight assertion and reporting layer, *Assertify*, which produces a compact and structured summary of test failures.

The objectives of this thesis are as follows:

- Analyse the structure and typical content of failing test output in beginner exercises, with particular attention to which elements are essential for interpreting the failure.
- Design and implement a custom test/assertion library that enforces a consistent reporting scheme (e.g., a compact summary of the `tested expression` and `expected` as well as `actual` value) on top of existing frameworks.
- Evaluate the resulting artifact with respect to the derived requirements and reflect on remaining limitations, thereby identifying technically feasible extensions and future improvements.

To achieve these objectives, the thesis presents *Assertify* as a wrapper around existing mechanisms for unit and property-based testing and discusses how the reporting format can be generalised, extended, and integrated into introductory teaching workflows.

1.2 Research Questions

This thesis seeks to answer the following research questions:

RQ1 (Problem characterization) Which structural characteristics of compiler and testing-framework outputs make failing tests difficult to interpret for novice programmers in introductory programming exercises?

RQ2 (Design requirements) Which requirements can be derived for novice-oriented test feedback with respect to consistency, cognitive load, and the actionability of failure information?

RQ3 (Reporting scheme) How can a uniform *tested expression/expected/actual* reporting scheme be defined such that it remains applicable across different kinds of tests (example-based unit tests and property-based tests)?

RQ4 (Technical feasibility) How can this reporting scheme be implemented in F# using Quotations and Unquote for expression capture and evaluation, while remaining compatible with existing tooling through wrappers around `Microsoft.VisualStudio.TestTools.UnitTesting` and `FsCheck`?

RQ5 (Results and discussion) To what extent does the implemented artifact (*Assertify* & *Checkify*) satisfy the derived requirements in practice, and what limitations and trade-offs emerge from the wrapper-based approach with respect to expression coverage, supported assertion forms, formatting constraints, and integration boundaries?

By addressing these questions, this thesis positions its contribution as the design and implementation of a test feedback layer that simplifies and standardises failure reporting for novice programmers, and as a critical reflection on how such a library can be extended and integrated into teaching contexts.

1.3 Thesis Structure

The structure of this thesis is as follows:

Background and Related Work reviews research on the difficulties novice programmers face when interpreting compiler and test diagnostics, and summarises prior approaches that aim to improve the structure and usefulness of error feedback. Building on these perspectives, the chapter motivates the thesis goal of presenting failures in a more consistent and cognitively simple format.

Problem Analysis examines typical compiler and testing-framework outputs as they appear in beginner exercises and characterises their technical properties, such as verbosity, inconsistency, and the mixing of essential information with framework internals. Based on these observations, the chapter derives design requirements for improved test failure reporting, with a stable *tested expression/expected/actual* summary as the primary user-facing interface.

Assertify (and Checkify): Design and Implementation presents the design rationale and implementation of *Assertify* as a lightweight test and assertion library that wraps existing testing infrastructures. It explains how a consistent `expected=actual` output format is produced, including the use of Quotations and Unquote for expression capture and evaluation, as well as wrappers around `Microsoft.VisualStudio.TestTools.UnitTesting` and `FsCheck`. The chapter also discusses implementation choices and their implications for extensibility.

Results and Discussion reports the results of the implemented approach and discusses them in relation to the derived requirements and research questions. The discussion focuses primarily on technical and qualitative properties achieved by the artifact (e.g., output consistency, coverage of supported assertion forms, and integration with unit and property-based testing workflows) and uses illustrative comparisons to baseline framework output to highlight differences in interpretability, while separating concrete observations from generalisable claims.

Conclusion concludes the thesis by summarising the contributions and revisiting the research questions in light of the presented results. It then outlines future work, including potential extensions within the wrapper-based approach (e.g., broader expression coverage, richer formatting layers, and tighter tooling integration) and longer-term directions that would require more substantial redesign beyond wrappers.

2 Background and Related Work

TODO

2.1 Challenges for Novice Programmers

TODO

2.2 Error Messages and Programming Education

TODO

2.3 Improving Feedback: Existing Aproches (Rust?)

TODO

3 Problem Analysis

This chapter analyses why the feedback produced by compilers and common testing infrastructures is frequently difficult to use in introductory programming settings. The focus is not on the correctness of the diagnostics themselves, but on how failure information is *structured*, *prioritised*, and *presented* to learners. Based on concrete examples from beginner exercises, the chapter identifies recurring shortcomings in existing output and derives design requirements for a novice-oriented reporting layer. These requirements motivate the subsequent design and implementation decisions, in particular the use of a stable *tested expression / expected / actual* summary as the primary interface for understanding failures.

3.1 Context and Assumptions

This thesis is situated in an introductory programming setting in which students solve small-to-medium programming exercises and receive automated feedback from compilation and testing. The tasks are designed to be assessable automatically, and correctness is primarily determined through a combination of example-based unit tests (covering typical and edge cases) and, where applicable, property-based tests (covering broader input spaces through generated test cases).

The problem analysis adopts the following context assumptions:

- **Novice audience:** The primary target group consists of first-semester students who have limited experience with debugging and limited familiarity with framework internals (e.g., stack traces, reflection-based test runners, or property-based test shrinking).
- **Feedback pipeline:** Students typically encounter feedback in two stages: (i) compilation diagnostics when code does not type-check or parse, and (ii) test failure output once code compiles and tests execute.
- **Tooling baseline:** Tests are executed using established infrastructures (unit testing and property-based testing libraries). The analysis treats these infrastructures as the baseline whose output is to be *interpreted by students*, not as systems to be redesigned from scratch.
- **Non-goals of this chapter:** This chapter does not attempt to redesign compiler diagnostics. Instead, it focuses on the *reporting problem* for test failures, while acknowledging that compiler errors remain a major part of the overall novice experience.

In addition, the analysis distinguishes between two kinds of information that appear in typical outputs:

- **Essential failure information** needed to understand *what* failed (e.g., the failing expression or behaviour, and the expected vs. actual outcome).
- **Auxiliary technical information** that can be useful for advanced debugging but often increases cognitive load for novices (e.g., full stack traces, internal framework calls, runner configuration details, reflection invocation frames).

Checkpoint (to fill with concrete course/tool details):

- TODO: Specify the course/exercise setting (language version, IDE/editor setup, where students see output: console, CI/autograder log, IDE test explorer, etc.).
- TODO: Summarise the types of tests used (example tests vs. random/property tests), including how failures are surfaced to students.
- TODO: Clarify what students are allowed to modify (submission file only vs. tests visible/hidden), as this affects what output is actionable.

3.2 Observed Issues in Existing Compiler/Test Output

This section characterises the structural properties of baseline outputs that contribute to poor interpretability for novices. The analysis is organised around recurring patterns, illustrated through representative examples from beginner exercises.

Issue 1: Essential information is fragmented or implicit

In many baseline test outputs, the information required to reconstruct the failing behaviour is not presented as a single, explicit summary. Instead, learners must infer it by combining multiple fragments (e.g., a test name, a partially printed expression, and a comparison line). When the *tested expression* is not explicitly shown, students may not be able to associate the failure with a specific function call in their submission, especially when a test method executes multiple assertions or multiple inputs.

Issue 2: Mixing core mismatch information with framework internals

Failure reports frequently interleave the core mismatch (expected vs. actual) with verbose framework-specific content such as stack traces, internal runner frames, and reflection invocation details. While such information can be valuable for expert debugging, it often dominates the output visually and makes it difficult for novices to identify the actionable part of the report.

Issue 3: Inconsistent structure across test types and frameworks

Even within a single course setup, students may encounter multiple output formats depending on whether a failure stems from:

(a) a unit-test assertion,

(b) a quotation-based assertion mechanism, or

(c) a property-based test with counterexample generation and shrinking.

If these formats differ substantially in vocabulary, ordering, and level of detail, students cannot form stable expectations about where to look for essential information.

Issue 4: Property-based testing output is informative but not pedagogically structured

Property-based testing frameworks can provide powerful information (e.g., a counterexample input and a shrunk minimal failing case). However, this information is often embedded in output that assumes familiarity with the property-testing workflow. In particular, the relationship between (i) the generated input, (ii) the shrunk counterexample, and (iii) the failing assertion is not always made explicit in a way that supports novice mental models.

Issue 5: Missing execution context for stateful or multi-step scenarios

For exercises involving stateful data structures or sequences of operations, a single expected/actual mismatch may be insufficient to understand the failure. If the failure occurs after a sequence of updates, students may need to know *what was executed before the failing check* in order to reproduce the bug. Baseline outputs typically do not preserve such operational context.

Checkpoint (examples and evidence to add):

- TODO: Insert 2–3 concrete baseline outputs from your own exercise set and annotate them (highlight: where is the tested expression? where is expected/actual? what is noise?).
- TODO: Add at least one example for each: (i) example-based unit test failure, (ii) quotation/assertion failure, (iii) property-based failure with shrinking.
- TODO: Provide a short "signal vs. noise" breakdown for each example (e.g., mark lines as essential vs. auxiliary).
- TODO: If you discuss multi-step/stateful tests, add a small scenario (e.g., linked list operations) and show precisely what context is missing in baseline output.

Mock-up: additional observations to validate with literature (not yet backed by your local examples)

The following patterns are frequently discussed in the broader literature on novice-facing diagnostics and may apply to your setting, but should be validated with either (i) your own collected outputs or (ii) clearly cited prior work:

- TODO: **Jargon and modality switching:** natural-language fragments mixed with code tokens can force frequent context switches during reading.
- TODO: **Redundant repetition:** repeating the same information in multiple forms (e.g., multiple representations of the same expression) can increase reading effort without improving comprehension.
- TODO: **Poor localisation:** messages may be accurate but point novices to symptoms rather than the likely source of misunderstanding.

3.3 Derived Requirements and Design Implications

Based on the issues above, this section derives requirements for novice-oriented test failure reporting. Each requirement is phrased as a constraint on the *user-facing report*, independent of any specific implementation technique. The subsequent design chapter then maps these requirements onto concrete mechanisms (e.g., expression capture, value extraction, and framework wrappers).

Requirements

R1: Stable summary structure. Every failing test should provide a compact, consistently ordered summary containing at least: (i) the tested expression (or an equivalent description of the behaviour under test), (ii) the expected value/condition, and (iii) the actual value/observed result.

R2: Explicit tested expression. The report must explicitly identify *what was executed* (e.g., a reconstructed function call) rather than relying on the test name or on implicit context.

R3: Expected vs. actual as the primary comparison. The expected and actual outcomes must be presented side-by-side (or in a tightly coupled pair) in a way that supports rapid visual comparison.

R4: Separation of essential and auxiliary information. Framework internals (stack traces, runner frames, reflection calls) must not obscure the essential summary. If included at all, such details should be clearly separated and optionally suppressible.

R5: Applicability across test paradigms. The same summary structure should apply to both example-based unit tests and property-based tests, even when additional information (e.g., counterexamples) is present.

R6: Structured representation of property-based counterexamples. If a property fails, the report must clearly indicate (i) the failing input, (ii) any shrunk/minimised counterexample, and (iii) how this input relates to the tested expression and observed mismatch.

R7: Support for contextual history when needed. For multi-step/stateful scenarios, the reporting layer should be able to attach an execution history (e.g., a sequence of operations) to make the failure reproducible and interpretable.

Design implications for Assertify/Checkify

The requirements above imply that a reporting layer must (a) gain access to the tested expression at runtime, (b) evaluate it to obtain the actual value, (c) retain or compute the expected value, and (d) serialise both into a stable output model that is independent of any particular baseline framework.

In the scope of this thesis, these implications motivate:

- **Expression capture:** obtain a representation of the tested expression that can be displayed to the user (and simplified where appropriate).
- **Value extraction:** evaluate expressions in a controlled way to compute actual values and to render values consistently.
- **Wrapper-based integration:** intercept failures from existing unit testing and property-based testing frameworks and replace or augment their output without requiring changes to the surrounding tooling.
- **Unified output model:** define a single report representation that can be produced for different assertion forms and test paradigms.

Checkpoint (traceability to later chapters):

- TODO: Add a small mapping table from issues → requirements → implementation mechanisms (one row per requirement).
- TODO: Decide which requirements are *must-have* vs. *nice-to-have* for the first version of Assertify.
- TODO: State explicitly which requirements are out of scope (e.g., improving compiler diagnostics beyond test reporting).

Completion Checklist for This Chapter

- Add at least one fully worked example for each failure category (compiler error, unit test failure, property-based failure).
- Add an explicit "signal vs. noise" annotation for each example output.
- Ensure every derived requirement is supported by at least one observed issue (traceability).

- Add short, properly cited connections to prior work on diagnostic usability and novice comprehension (do not overclaim; separate observation from generalisation).
- Verify terminology consistency with later chapters (e.g., always use *tested expression* / *expected* / *actual* in the same order).

4 Assertify (and Checkify): Design and Implementation

TODO

4.1 Design Goals and Principles

TODO

4.2 Output Model: Representing Tests

TODO

4.3 Mechanisms: Expression Capture and Value Extraction

TODO: - leftSide, rightSide extraction, blablabla - CaptureRunner

4.4 Reporting: Formatting and Integration with the Test Pipeline

TODO: - With output comparison

5 Results and Discussion

TODO: - Technical validation: what works, coverage, limitations - Structured comparison: criteria-based analysis, referencing earlier examples

5.1 Research Question Coverage and Requirement Satisfaction

TODO

5.2 Artifact Outcomes

TODO

5.3 Key Findings and Interpretations

TODO

5.4 Strengths and Limitations

TODO

6 Conclusion

TODO

6.1 Summary

TODO

6.2 Future Work

TODO: - Optimizations - Useful Evaluation/Data once the code is running smoothly on ExClaim - FSharp.Compiler.Services to reflect on what went wrong: - A combination of FSharp.Compiler.Services and FParsec would be able to produce better errors for majority of failures - TODO DATA OF TOP 10 ERROR CODES !!!!! - Generalization via MiniLib: - Including student friendly helper modules/types/etc - Test Library with configurable verbosity to suit multiple levels of learning progress - Maybe implement it as a full Test Framework - Combine with VS Code and code-server?

6.3 Final Remarks

TODO

List of Figures

List of Listings

List of Tables

My Code

My Ideas